

C# Coding Interview Questions & Problems

Complete Reference Guide • 80+ Questions • 10 Sections

SECTION 1: String Manipulation

Q1. Reverse Characters in a String

- a. Prepend Approach
- b. String Buffer (StringBuilder)
- c. Swap (two-pointer in-place)

Q2. Reverse Words in a String

Q3. Find Occurrence of Each Character in a Given String

Q5. Check if a String is a Palindrome

■ Tip: Compare $s[i]$ with $s[n-1-i]$ or use `string.Reverse()`

Q7. Anagram Code – Check if Two Strings are Anagrams

■ Approach: Sort both strings or use a frequency dictionary

Q9. Find the First Non-Repeating Character

■ Approach: Use `Dictionary<char,int>` then scan for `count==1`

Q10. Remove Duplicates from a String

Q13. Extract the Surname from a Full Name String

Use `string.Split(' ')` and take the last token

Q14. Find the Longest Substring Without Repeating Characters

Sliding Window approach using `HashSet`

■ Example: "eghghhg" → 3 ("egh") | "substring" → 8 ("ubstring")

Q15. Remove All White Spaces from a String

`s.Replace(" ", "")` or `string.Concat(s.Split())`

Q16. Find the Longest Palindromic Substring

Expand-around-center approach: $O(n^2)$

Q30. Valid Parentheses

Use a `Stack<char>`. Push opening brackets, pop and match on closing.

■ Valid: '()[]{}' → true | '(D)' → false

Q51. Print All Substrings of a String

Two nested loops: outer i from $0..n$, inner j from $i..n$

■ Input: "abcd" → a ab abc abcd b bc bcd c cd d

Q52. Count Vowels and Consonants in a String

Q53. Check if Two Strings are Rotations of Each Other

Concatenate $s1+s1$, then check if $s2$ is a substring

■ Example: "abcde" and "cdeab" → true

Q54. Title Case a Sentence

Use `CultureInfo.CurrentCulture.TextInfo.ToTitleCase(s)`

Q55. Compress a String (Run-Length Encoding)

Input: "aabbcccc" → Output: "a2b3c4"

SECTION 2: Arrays & Numbers

Q4. Find Two Numbers in an Array That Add Up to a Target

HashSet approach: $O(n)$ time, $O(n)$ space

■ For each num, check if (target-num) exists in the set

Q6. Reverse an Array

Two-pointer swap: $i=0, j=n-1$ until $i < j$

Q8. Move All 0's to the End of an Array

Two-pointer: keep a write pointer for non-zero elements

Q11. Swap Two Numbers

Without temp: $a=a+b; b=a-b; a=a-b$

With XOR: $a^=b; b^=a; a^=b$

Q12. Reverse a Number Without Converting to String

Extract digits using $\% 10$, build reversed number with $*10$

Q17. Print Array in Spiral / Pattern

`int[] arr = {1,2,3,4,5,6};` Output: //1 2 3 4//5 6 1 2

Q18. Left/Right Rotate an Array by K Places

Reverse sub-arrays: `reverse(0,k-1)`, `reverse(k,n-1)`, `reverse(0,n-1)`

Q21. Find the Maximum Number in an Array

Q27. Dutch National Flag / Sort Colors (0s, 1s, 2s)

Three-pointer: lo, mid, hi. Partition in single pass.

■ Input: `[2,1,2,0,1,0,1,0,1]` → Output: `[0,0,0,1,1,1,1,2,2]`

Q29. Maximum Sum of a Contiguous Subarray of Size k (Sliding Window)

Compute first window sum, then slide: add right, remove left

■ Input: `[2,1,5,1,3,2]`, $k=3$ → Output: 9 (subarray `[5,1,3]`)

Q56. Find Minimum Value in an Array

Q57. Remove Even Integers from an Array

Use LINQ: `arr.Where(x => x % 2 != 0).ToArray()`

Q58. Find Duplicate Elements in an Array

`GroupBy(x=>x).Where(g=>g.Count(>1)).Select(g=>g.Key)`

■ Input: `[1,2,3,2,4,5,5]` → Output: 2, 5

Q59. Find the Second Largest Element in an Array

Sort descending and return index 1, or scan with two variables

Q60. Kadane's Algorithm – Maximum Subarray Sum

`currMax = max(A[i], currMax+A[i]); globalMax = max(globalMax, currMax)`

■ Input: `[-2,1,-3,4,-1,2,1,-5,4]` → Output: 6 (subarray `[4,-1,2,1]`)

Q61. Merge Two Sorted Arrays Without Extra Space

Q62. Find Missing Number in Array 1 to N

Sum formula: $\text{expected} = n*(n+1)/2 - \text{actual sum}$

Q63. Product of Array Except Self

Build prefix and suffix product arrays without division

Q64. Rotate a Matrix 90 Degrees Clockwise

Transpose the matrix, then reverse each row

Q65. Binary Search in a Sorted Array

$O(\log n)$: compare mid, discard half on each step

SECTION 3: Math & Recursion

Q19. Prime Numbers – Check and List Primes

Trial division up to $\sqrt{n}$ for single check

Sieve of Eratosthenes for listing all primes up to N

Q20. Factorial Using Recursion

Base case: $\text{Factorial}(0)=1$; Recursive: $n * \text{Factorial}(n-1)$

Q22. Fibonacci Series

Recursive, Iterative, and Dynamic Programming approaches

■ *Output: 1, 1, 2, 3, 5, 8, 13, 21...*

Q24. Next Number in Pattern: 1, 3, 7, 13, ...

Differences: 2, 4, 6, 8 → next difference = 10 → answer = 21

Q66. Power Function – Implement Math.Pow(x, n)

Fast exponentiation: divide n by 2 recursively → $O(\log n)$

Q67. GCD and LCM of Two Numbers

GCD: Euclidean algorithm $\text{gcd}(a,b) = \text{gcd}(b, a\%b)$

LCM: $\text{lcm}(a,b) = a*b / \text{gcd}(a,b)$

Q68. Sum of Digits of a Number

While $n>0$: $\text{sum} += n\%10$; $n /= 10$

Q69. Armstrong Number Check

$153 = 1^3 + 5^3 + 3^3 \rightarrow \text{Armstrong}$

■ *Sum of each digit raised to power of digit-count equals the number*

Q70. Perfect Number Check

Sum of proper divisors equals the number. Example: $28 = 1+2+4+7+14$

Q71. Count Set Bits in an Integer (Brian Kernighan's Algorithm)

$n = n \& (n-1)$ removes the lowest set bit each iteration

SECTION 4: Data Structures (Linked List, Stack, Queue)

Q72. Implement a Singly Linked List

Node class with data + next pointer

Methods: InsertAtHead, InsertAtTail, Delete, Print

Q73. Reverse a Linked List

Iterative: $\text{prev}=\text{null}$, $\text{curr}=\text{head}$, $\text{next}=\text{curr.next}$; reverse pointers

Q74. Detect a Cycle in a Linked List (Floyd's Algorithm)

Slow pointer moves 1 step, fast pointer moves 2 steps
If they meet → cycle exists

Q75. Remove Duplicates from a Linked List

Use a HashSet to track seen values while traversing

Q76. Merge Two Sorted Linked Lists

Q77. Find the Middle of a Linked List

Slow/fast pointer: slow moves 1, fast moves 2 → slow is at middle when fast reaches end

Q78. Implement a Stack Using Arrays / LinkedList

Operations: Push, Pop, Peek, IsEmpty — all O(1)

Q79. Implement a Queue Using Two Stacks

Enqueue to stack1; Dequeue: move all to stack2, pop top

Q80. Implement a Stack That Returns Minimum in O(1)

Use auxiliary min-stack; push current min alongside each element

Q81. Implement a Queue Using Arrays (Circular Queue)

Q82. Balanced Parentheses Using Stack

Push '(', pop when ')' found, check top matches

Q83. Implement LRU Cache

LinkedList<T> + Dictionary<K, LinkedListNode>: O(1) get and put

■ Evict least-recently-used item when capacity is full

Q84. Priority Queue – PriorityQueue<T,P> (.NET 6+)

Dequeues element with smallest priority value first (min-heap internally)

■ Use case: Dijkstra's shortest path, task scheduling

SECTION 5: Trees & Graphs

Q85. Implement a Binary Search Tree (BST)

Insert, Search, Delete operations

Left child < parent < right child

Q86. Find the Minimum / Maximum Value in a BST

Min: go left until no left child

Max: go right until no right child

Q87. Find the Height of a Binary Tree

Recursive: $1 + \max(\text{Height}(\text{left}), \text{Height}(\text{right}))$

Q88. In-order, Pre-order, Post-order Traversal

In-order (L-Root-R): gives sorted output for BST

Pre-order (Root-L-R): used to recreate the tree

Post-order (L-R-Root): used for deletion

Q89. Check if a Binary Tree is a BST

Q90. Find the In-order Successor in a BST

If right subtree exists → leftmost node in right subtree

Else → last ancestor for which the node is in the left subtree

Q91. Lowest Common Ancestor (LCA) in a BST

If both p and $q < \text{node}$ → go left; both $> \text{node}$ → go right; else → LCA

Q92. Depth First Search (DFS) on a Graph

Use Stack (iterative) or recursion; mark visited nodes

Q93. Breadth First Search (BFS) on a Graph

Use Queue; process level by level

Q94. Detect a Cycle in a Directed Graph

DFS with a recursion stack / visited colour (white/grey/black)

Q95. Find the Shortest Path (Dijkstra's Algorithm)

PriorityQueue of (distance, node); relax edges greedily

SECTION 6: Sorting & Searching Algorithms

Q96. Bubble Sort

Repeatedly swap adjacent elements if out of order — $O(n^2)$

Q97. Selection Sort

Find minimum in unsorted portion, swap to front — $O(n^2)$

Q98. Insertion Sort

Insert each element into its correct position in sorted portion — $O(n^2)$

Q99. Merge Sort

Divide in half, sort each half, merge — $O(n \log n)$ stable sort

Q100. Quick Sort

Choose pivot, partition, recurse — $O(n \log n)$ average, $O(n^2)$ worst

■ *Key interview question: how to choose pivot to avoid worst case*

Q101. Counting Sort / Radix Sort

Counting sort: $O(n+k)$ for small integer range

Radix sort: sort digit by digit, LSD to MSD

Q102. Binary Search (Iterative and Recursive)

Only works on sorted arrays — $O(\log n)$

Q103. Search in Rotated Sorted Array

Modified binary search: determine which half is sorted, then narrow

■ *Example: [4,5,6,7,0,1,2], target=0 → index 4*

SECTION 7: Patterns & Output

Q23. Basic Star Pattern Examples

Print various patterns using nested for loops.

• *Right triangle* • *Inverted triangle* • *Pyramid* • *Diamond* • *Pascal's triangle*

Q104. Print Number Pattern

1 / 1 2 / 1 2 3 / 1 2 3 4 — two nested loops, inner prints 1..i

Q105. Print a Multiplication Table

for i 1..10: for j 1..10: Console.Write($i*j$)

SECTION 8: SQL / Database

Q25. Find Duplicate Records in a Table

SELECT col, COUNT(*) FROM table GROUP BY col HAVING COUNT(*) > 1

Q26. Find Nth Highest Salary

Using DENSE_RANK(): SELECT * FROM (SELECT *, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rn FROM Emp) t WHERE rn = N

■ Alternative: Use ROW_NUMBER() or correlated subquery

Q106. Delete Duplicate Rows Keeping One

WITH CTE AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY col ORDER BY id) rn FROM t) DELETE FROM CTE WHERE rn > 1

Q107. Find Employees with Salary Greater Than Their Manager

Self-join: SELECT e.Name FROM Emp e JOIN Emp m ON e.ManagerId = m.Id WHERE e.Salary > m.Salary

Q108. Difference Between INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN

Q109. What is an Index? When Should You Use One?

Speeds up SELECT but slows INSERT/UPDATE. Use on frequently queried columns.

Q110. What is a Stored Procedure vs a Function?

SP: can modify state, no return value required

Function: returns value, can be used in SELECT; cannot have side effects

SECTION 9: C# Language & OOP Concepts

Q31. OOP Concepts: Encapsulation, Abstraction, Inheritance, Polymorphism

Encapsulation: hide internal state (private fields + public properties)

Abstraction: expose only what is necessary (abstract class / interface)

Inheritance: derived class reuses base class code (: BaseClass)

Polymorphism: method behaves differently based on runtime type (virtual/override)

Q32. Abstract Class vs Interface

Abstract class: can have implementation, single inheritance only

Interface: contract only (C#8+ can have default impl.), multiple implementation

Q33. Value Types vs Reference Types

Value: stored on stack (int, double, struct, enum) — copied on assignment

Reference: stored on heap (class, string, array) — copied by reference

Q34. Boxing and Unboxing

Boxing: int → object (heap allocation, slower)

Unboxing: object → int (must match exact type)

■ *object a=123; object b=123; a==b → False (different references!)*

Q35. Delegates, Events, and Lambda Expressions

Delegate: type-safe method reference (like a function pointer)

Multicast delegate: references multiple methods, all called in order

Event: delegate with restricted access — only owner can invoke

Q36. LINQ Queries

Method syntax: list.Where(x=>x.Salary>50000).OrderBy(x=>x.Name)

Query syntax: from e in list where e.Salary>50000 select e

Both compile to the same IL — method syntax more flexible in practice

Q37. async / await and Task Parallel Library

async/await frees thread during I/O without blocking

Avoid async void (except event handlers) — use async Task
Avoid .Result / .Wait() — causes deadlocks on UI thread
Use ConfigureAwait(false) in library code

Q38. Exception Handling Best Practices

catch specific exceptions before general ones
Use finally for cleanup (close DB connections, file handles)
Custom exceptions: inherit from Exception, add constructor overloads

Q39. Garbage Collection and IDisposable

GC manages heap; runs in generations (Gen0, Gen1, Gen2)
Implement IDisposable + using statement for unmanaged resources
Use SafeHandle or finalizer for critical native resources

Q40. Collections: List, Dictionary, HashSet, Queue, Stack

List<T>: dynamic array, O(1) index access
Dictionary<K,V>: hash table, O(1) average get/set
HashSet<T>: unique values, O(1) Contains
Queue<T>: FIFO | Stack<T>: LIFO

Q41. String vs StringBuilder

string is immutable — concatenation in loop = O(n²) allocations
StringBuilder is mutable — use for building strings in loops (O(n))

Q42. Nullable Types and Null-Safe Operators

int? x = null; (Nullable<int>)
?? null coalescing | ??= null coalescing assignment
?. null-conditional (member access) | ?[] null-conditional (index)

Q43. Extension Methods

static class, static method, first param is 'this Type name'
■ Example: `public static bool IsPalindrome(this string s) { ... }`

Q44. Dependency Injection in .NET

IoC: class receives dependencies instead of creating them
ASP.NET Core DI: services.AddSingleton / AddScoped / AddTransient
Singleton: one instance | Scoped: per request | Transient: per call

Q45. Design Patterns in C#

Singleton: private constructor + static Instance property
Factory: abstract creation — returns base type
Repository: data access abstraction layer
Observer: IObservable<T> / IObservable<T> or events

Q46. Generic Methods and Classes

public static void Swap<T>(ref T a, ref T b) { T t=a; a=b; b=t; }
Constraints: where T : class / struct / new() / IComparable<T>

Q47. Reflection

Type t = typeof(MyClass); or obj.GetType();
Get properties: t.GetProperties() | Invoke method: t.GetMethod().Invoke()

Q48. Record Types (C# 9+)

Immutable by default, value-based equality, with-expressions for copy
record Person(string Name, int Age); — positional record

Q49. Pattern Matching

switch expression (C#8+): `x switch { int i => i*2, string s => s.Length, _ => 0 }`
is pattern: `if (obj is string s && s.Length > 3)`

Q50. Memory: Stack vs Heap

Stack: local variables, method frames — fast, auto-freed
Heap: reference type objects — GC managed, slower allocation

Q111. IEnumerable vs IQueryable

IEnumerable: in-memory LINQ, executes in C#
IQueryable: translated to SQL, executes in database — use for EF queries

Q112. Span<T> and Memory<T>

Span<T>: stack-only, zero-allocation slice over memory
Use for high-performance parsing without allocating substrings

Q113. Partial Classes and Partial Methods

Class split across files, combined at compile time
Common in WinForms/WPF designer-generated code

Q114. Covariance and Contravariance

`IEnumerable<Derived> → IEnumerable<Base>` (covariant: out keyword)
`Action<Base> → Action<Derived>` (contravariant: in keyword)

Q115. Thread Safety: lock, Monitor, Mutex, SemaphoreSlim

`lock(obj)`: ensures single thread in critical section
SemaphoreSlim: limit concurrent access to N threads
Avoid deadlocks: always acquire locks in the same order

Q116. Entity Framework Core – Code First vs Database First

Code First: define C# models → EF generates DB schema via migrations
Database First: existing DB → EF scaffolds C# models
Loading: Eager (Include), Lazy (virtual nav props), Explicit (Load())

SECTION 10: Dynamic Programming & Advanced Algorithms

Q117. Climbing Stairs Problem

You can climb 1 or 2 steps. How many ways to reach step n?
DP recurrence: $dp[i] = dp[i-1] + dp[i-2]$ (same as Fibonacci)
■ Input: $n=4 \rightarrow$ Output: 5

Q118. 0/1 Knapsack Problem

$dp[i][w] = \text{max profit using first } i \text{ items with capacity } w$
Choice: include item ($dp[i-1][w-wt[i]] + val[i]$) or exclude ($dp[i-1][w]$)

Q119. Longest Common Subsequence (LCS)

$dp[i][j] = \text{LCS of } s1[0..i] \text{ and } s2[0..j]$
If chars match: $dp[i][j] = dp[i-1][j-1] + 1$; else $\max(dp[i-1][j], dp[i][j-1])$

Q120. Coin Change Problem – Minimum Coins

$dp[\text{amount}] = \text{min coins to make that amount}$
For each coin: $dp[i] = \min(dp[i], dp[i-\text{coin}] + 1)$
■ Input: $\text{coins}=[1,5,6,9], \text{amount}=11 \rightarrow$ Output: 2 (coins: 5+6)

Q121. House Robber Problem

Cannot rob two adjacent houses. Max money robbed?

$dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$

Q122. Two Sum – All Pairs

Return all index pairs where $\text{arr}[i] + \text{arr}[j] == \text{target}$

HashMap: store complement index; handle duplicates

Q123. Trapping Rain Water

For each bar: $\text{water} = \min(\text{maxLeft}, \text{maxRight}) - \text{height}[i]$

Use prefix arrays or two-pointer approach for $O(n)$

■ Input: $[0,1,0,2,1,0,1,3,2,1,2,1] \rightarrow$ Output: 6

Q124. Find All Permutations of a String

Backtracking: fix position i , swap with $i..n$, recurse, swap back

Q125. Subset Sum – Find All Subsets

Backtracking or bitmask: 2^n subsets for n elements

Q126. Word Break Problem

Given string and dictionary, can the string be segmented into dictionary words?

$dp[i] = \text{true}$ if $s[0..i-1]$ can be broken into words

■ Input: $s = \text{"leetcode"}, \text{dict} = [\text{"leet"}, \text{"code"}] \rightarrow \text{true}$

Quick Reference: Big-O Complexity Cheat Sheet

Algorithm / Structure	Time (Avg)	Time (Worst)	Space
Array access	$O(1)$	$O(1)$	$O(n)$
Binary Search	$O(\log n)$	$O(\log n)$	$O(1)$
Bubble / Selection / Insertion Sort	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Dictionary / HashSet lookup	$O(1)$	$O(n)$	$O(n)$
BST search (balanced)	$O(\log n)$	$O(n)$	$O(n)$
Linked List access	$O(n)$	$O(n)$	$O(n)$
Linked List insert/delete (head)	$O(1)$	$O(1)$	$O(n)$
BFS / DFS on Graph	$O(V+E)$	$O(V+E)$	$O(V)$
Dijkstra (PriorityQueue)	$O((V+E) \log V)$	$O((V+E) \log V)$	$O(V)$